

Design and Build a Balancing Robot

Name*¹

¹Research School of Physics, Australian National University, Canberra ACT, 2601, Australia

Abstract—

1. EXECUTIVE SUMMARY

Executive Summary

2. INTRODUCTION

2.1. Background

Background

2.2. Requirements Review

The primary objective of this project is to build an autonomous self-balancing robot which is able to move to or keep itself around a designated position. To achieve this, the listed specific requirements are as follows:

- Top-level requirements:
 - The robot must maintain its balanced position without any falling cases.
 - If the robot is perturbed externally (e.g., a deliberated push, or an obstacle collided), it must be able to return balanced within 2s. The perturbation is defined as changing its pitch angle up to 15°.
 - Without perturbation, the robot must balance itself in a small range. It must not move more than 0.5m in 1s.
- Electronic components:
 - The robot must be able to sense its angle or velocity at > 10Hz. The resolution of angle must be < 0.5°.
 - The robot must take action within 0.1s after a controlling signal is sent.
 - The electronics must be stable during its runtime.
 - After turning on, the robot must work entirely independently, including power supply, signal processing, and controlling.
- Physical structure:
 - The structure must be stable enough that the position of mass center doesn't change.
 - The chassis must be sturdy, which can resist damage when the robot falls in some cases.
- Advanced requirements:
 - The robot can be able to fully control its state, including facing direction and moving speed.
 - The robot can follow a designated path without obvious drift.
 - The robot can smartly respond to different situations: in peaceful environments, being slightly perturbed, being strongly perturbed, set to move to and keep around a position.

3. TECHNICAL DETAILS AND ANALYSIS

3.1. Basic Ideas

Fig. 1 shows the flowchart of the basic balancing idea of the robot. Generally, it shows the realization of the basic balancing skills. Any other advanced skills can be add to these parts. Reading multiple types of states (e.g., pitch angle, yaw angle, and x-acceleration) can be

add in the "Read State" process. "Check States" decision can be used to assess the situation of the robot. "Feedback Controller" process can handle different controllers, such as PID controller. "Take Action" process controls the states of the robot.

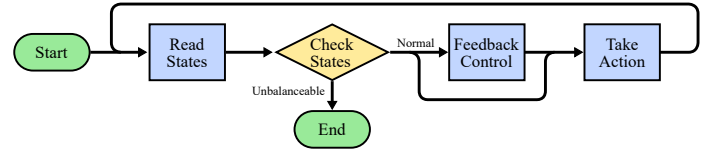


Figure 1. Flowchart of basic balancing logic. After turning on, the robot keeps reading its state, such as the pitch angle or the angular velocity of y-axis. If it's not strongly unbalanced, feedback controllers are used to generate an output from the state. The output is used to control the robot, such as moving forward or turn left. The feedback loop can balance the robot theoretically.

We started from developing the basic balancing skill. When this first stage was completed successfully, we would try to realize the advanced requirements by upgrading the existed processes or decisions instead of rebuild a new structure. In this way, the development would be controllable and the progress would be clear. However, a new structure can be required if we want to develop a more complex logic.

3.2. Electronic Components

To achieve the main goal, the electronic components should contain these necessary parts: a state sensor, a central processor, an action driver, and power supply.

3.2.1. The state sensor

We use MPU-6050 inertial measurement unit (IMU) as the state sensor. It consists of a triple-axis gyroscope and a triple-axis accelerometer. Using the angular velocities and linear accelerations, we can get or calculate most of the important states of robot. The key parameters of MPU-6050 are listed in Table 1:

Parameter	Value	Unit
Gyroscope Full-scale Range	±250	°/s
Gyroscope Sensitivity	131	LSB/(°/s)
Gyroscope Noise Spectral Density	0.005	°/s/√Hz
Gyroscope Output Rate	8000	Hz
Accelerometer Full-scale Range	±2	g
Accelerometer Sensitivity	16384	LSB/g
Accelerometer Noise Spectral Density	400	μg/√Hz
Accelerometer Output Rate	1000	Hz
Supported Communication	I ² C	
Communication Speed	400	kHz
Clocking Speed	32.768k ~ 19.2M	Hz
Power Input Voltage	2.375 ~ 3.46	V

Table 1. The key parameters of MPU-6050 inertial measurement unit [1]

The best resolution of angular velocity can reach 7.6×10^{-3} °/s, and the best resolution of acceleration can reach 61×10^{-6} g. Ignoring the noise, these resolutions are totally sufficient for the requirements. Both the sampling rate and communication speed satisfy our requirements. The only disadvantage is the sensor noise or drift. Considering the noise, we can try to process the signal (such as low-pass filter or quaternion) to get high enough resolution.

*email@anu.edu.au

3.2.2. The central processor

Arduino UNO is chosen as the processor. We can use it to get data from the IMU, process the data, and control the action driver. Its key parameters are listed in Table 2:

Parameter	Value	Unit
# Digital I/O Pins	14	
# Analog Input Pins	6	
# PWM Pins	6	
Supported Communication	$I^2C, SPI, UART$	
Clocking Speed	16	MHz
Power Output Voltage	3.3, 5	V
Power Input Voltage	7 ~ 12	V

Table 2. The key parameters of Arduino UNO [2]

The number of digital, analog and PWM pins are enough for the wiring connecting the processor to the sensor and the action driver. The high processing speed is ideal for our control logic. Using I^2C communication and 3V3 output power, it can math the MPU-6050 very well. However, the power output voltage may not enough for some action drivers. So a shared power supply may be required.

3.2.3. The action driver

We choose to use a L298N motor driver and two geared DC motors. The key parameters of the L298N motor driver are listed in Table 3:

Parameter	Value	Unit
Digital Input Pins	$IN1 + IN2, IN3 + IN4$	
PWM Input Pins	ENA, ENB	
Logic Input Voltage	5	V
Power Output Voltage	3.2 ~ 40	V
Power Input Voltage	5 ~ 35	V

Table 3. The key parameters of L298N motor driver [3]

The motor driver allows PWM input, which matches well with the Arduino UNO. The digital input pins can be used to control the direction of motors, while the PWM input pins can be used to control the speed. The required voltage is high as expected, so a good power supply is needed. However, by experiments, we found that the motors can keep still when the speed signal is small, which is likely caused by the resistance. Moreover, the two motors have a little asymmetry, which makes the robot yaw. Therefore, stepper motors can be required when we need to do precise control or measurement.

3.2.4. Power supply

We need to supply power to both the Arduino UNO and the motor driver. Therefore, two 800mAh 3.7V rechargeable Li-ion cells are used. For reusability, we use a breadboard to transfer the power from batteries to the two components instead of soldering the wires. The safe charged voltage of Li-ion is 2.8 ~ 4.2 V. By testing, the batteries need to be recharged after working for about 10 hours.

3.2.5. Wiring structure

As mentioned above, the electronic components used are:

- MPU-6050 IMU
- Arduino UNO
- L298N motor driver
- Geared DC motors
- Li-ion Batteries
- Breadboard

The wiring method is shown in Fig. 2.

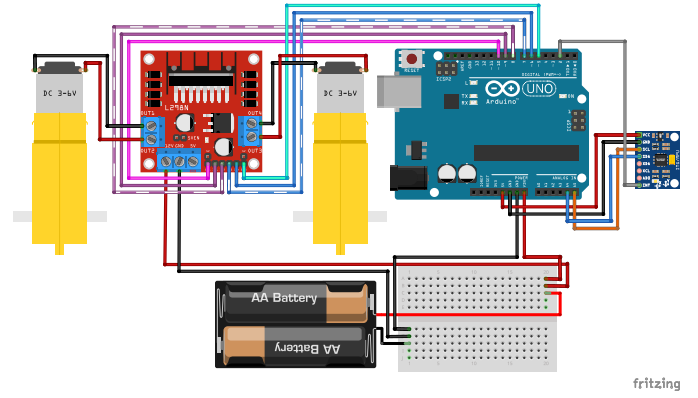


Figure 2. Wiring diagram of the electronic components. The battery and breadboard are used to power the Arduino UNO and L298N motor driver. UNO communicates with MPU-6050 inertial measurement unit (IMU) using I2C protocol, and sends analog signal to the motor driver using pulse width modulation (PWM). The motor driver controls 2 motors.

3.3. Mechanical Design

3.3.1. Physical model analysis

Fig. 3 shows the simplified physical model of the robot. We can use Lagrangian mechanics to analyze the relationship between F and θ , x . The system Lagrangian and Euler-Lagrange equations are given by:

$$L = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m[(\dot{x} + l\dot{\theta}\cos\theta)^2 + (l\dot{\theta}\sin\theta)^2] - mgl\cos\theta \quad (1)$$

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{x}} - \frac{\partial L}{\partial x} = F \Rightarrow (M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F \quad (2)$$

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}} - \frac{\partial L}{\partial \theta} = 0 \Rightarrow \ddot{x}\cos\theta + l\ddot{\theta} - g\sin\theta = 0 \quad (3)$$

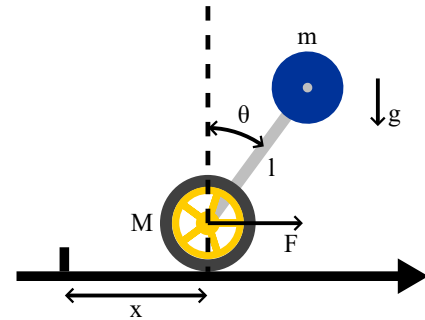


Figure 3. A simplified physical model of the robot. Constrain the movement in 1 dimension. The model consists of a wheel with mass M , a body mass center m , and a light stick connecting these two with length l . The action of wheel is simplified as a force F . Assume the robot is at position x and its pitch angle is θ .

Usually, we care about the dynamic of θ , which can be derived from Equation 2 and Equation 3:

$$\ddot{\theta} = \frac{(M+m)g\sin\theta - F\cos\theta}{(M+m\sin^2\theta)l} - \frac{m\dot{\theta}^2\sin 2\theta}{2(M+m\sin^2\theta)} \quad (4)$$

According to the model Fig. 3 and this Equation 4, we can analyze some cases:

- The robot is perfectly at the balanced state, but a perturbation happens before the controller realizes it (i.e., $\theta \gtrsim 0$, $F = \dot{\theta} = 0$). In this case, $\ddot{\theta} \approx \frac{(M+m)g\sin\theta}{lM}$. To save time for the controller to

realize before θ becomes too large, we want $\ddot{\theta}$ to be as small as possible, which means we need to use small m and/or large l .

- The robot is at an unbalanced but stable state (i.e., $\theta > 0$, $\ddot{\theta} \approx \dot{\theta} \approx 0$). To keep a constant θ , it's necessary to make $F = (M + m)g \tan \theta$. However, because of the limit of motor power, F can be maintained for only a short time before the large speed brings large resistance. So we should make $F > (M + m)g \tan \theta$ to bring back balance before the saturation of motors.
- The robot is severely unbalanced (i.e., $\theta > 0$, $\dot{\theta} > 0$). In this case, the only possible way to balance the robot is to make $\ddot{\theta}$ as negative as possible (i.e., $\ddot{\theta} \ll 0$). To achieve this, we need to use a combination of large $F > (M + m)g \tan \theta$ and small l .
- The robot is inverted (i.e., $90^\circ < \theta < 270^\circ$ ($l < R_{\text{wheel}}$), $l = 0$, or $m = 0$). Now the robot is impossible to return to the desired balanced state (or the balanced state can't be defined), but becomes permanently stable.

In conclusion, a motor being able to produce huge F is wanted for all the cases. Similarly, it's good to have small m , M , and/or $\frac{m}{M}$. Large l is suitable for peaceful environment and high balance performance, while small l or inverted robot is suitable for severe environment and high controllability.

3.3.2. Chassis

We use CAD (Autodesk Fusion) to design the chassis model, and 3D-print the model. The chassis is designed with many components in the experimental stage to make some components easier to update. These components are assembled using M3 bolts and nuts. For the final design, all the components are combined into a whole one to make the chassis stable.

In Fusion, the chassis is designed with the help of the models of other electronic components. Official datasheets of the electronic components are also used. In this way, it's precise and accurate to design the size and position of chassis components and through holes. One of our CAD model designs is shown in Fig. 4. It has high mass center, which can perform good balancing skill but lack controllability.

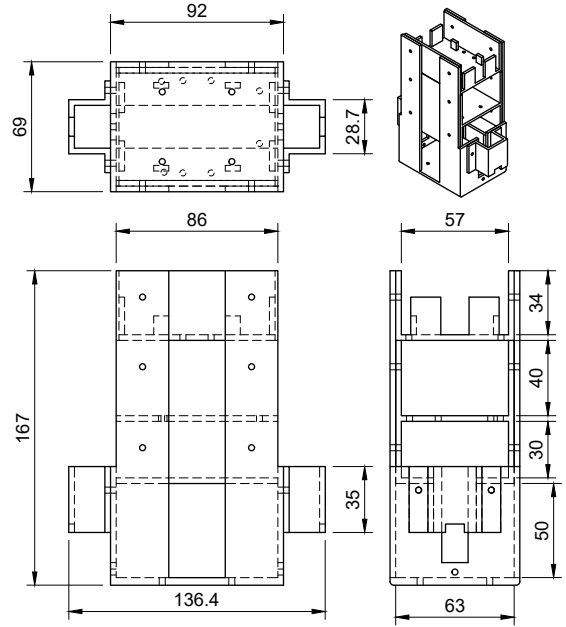
PrusaSlicer is used to convert CAD models to 3D-printing code. The printer uses PLA as the filament with diameter of 1.75mm. To make our chassis sturdy, we use layer height of 0.3mm. The vertical shells is set to 4 perimeters, and the horizontal shells is set to 4 top layers and 4 bottom layers. Since most of our chassis consist of plates of 3mm, these parameters are sufficient to make our chassis solid. The fill density is set to 15%. The only disadvantage is that solid chassis may increase the mass. We compensated it by change the height of mass center by a larger value.

3.4. Signal Processing

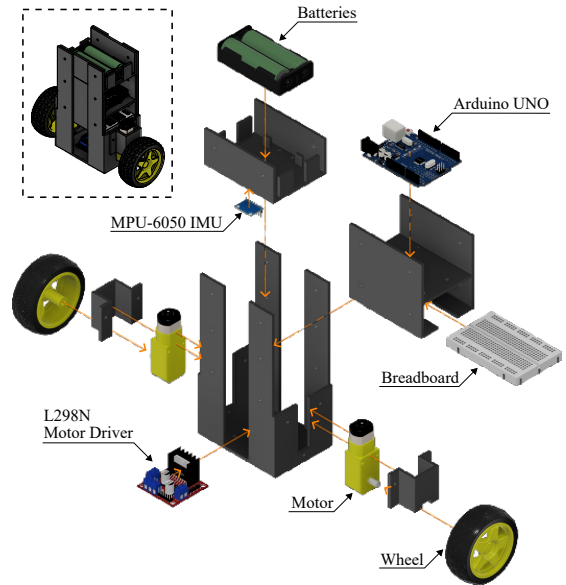
3.4.1. IMU data processing

To get the robot state, we need to process the data from the IMU. The IMU contains a 3-axis gyroscope used to measure angular velocity and a 3-axis accelerometer used to measure acceleration. One important thing we can do is to get the angle (yaw, pitch, roll) from these values. Simply, we can integral the 3 angular velocities ($\omega_x, \omega_y, \omega_z$) to get the corresponding Euler angle. But it suffers from Gimbal Lock (for example, the results are the same whether it yaws by 30° and then pitches by 90° , or pitches by 90° and then rolls by 30° , which makes it difficult to determine the true yaw or roll angle). Because of this, we use quaternion $q = (q_0, q_1, q_2, q_3)$ to represent the angle state, which indicates the current state that is equivalent to rotate the initial state around an axis (q_1, q_2, q_3) by an amount of q_0 . The current quaternion is given by:

$$q_n = q_{n-1} + \frac{\Delta t}{2} \cdot q_{n-1} \otimes \omega \quad (5)$$



(a) Front-top-side view



(b) Robot components and assembly

Figure 4. A chassis model design.

Another method to get quaternion is using the accelerations (a_x, a_y, a_z). Because there is a constant gravity of $\approx 9.8m \cdot s^{-2}$, we can know the quaternion by comparing the direction of measured acceleration with the true gravity. However, this method is noisy, while using gyroscope can produce drift over a long time. Therefore, we can use a weighted average of these two methods.

There are many states we can calculate from the quaternion (Most calculations can be done by the IMU module or some libraries such as `izcdevlib/Arduino/MPU6050`). For example:

$$\text{Pitch: } \theta = \arcsin(2(q_0q_2 - q_3q_1)) \quad (6)$$

$$\text{Yaw: } \psi = \arctan \frac{2(q_0q_3 + q_1q_2)}{1 - 2(q_2^2 + q_3^2)} \quad (7)$$

$$\text{Acceleration: } a_{\text{world}} = q \otimes (0, a_x, a_y, a_z) \otimes q^{-1} \quad (8)$$

In this way, we can get the tilt angles or accelerations much more precisely and unambiguously. However, to get the displacement of the robot, we need to integrate the acceleration twice, which can bring huge noise and drift. This disadvantage is mostly caused by the hardware, which is hard to improve by software. Therefore, we can choose to calculate the displacement from the signal used to control the speed of motors.

3.4.2. PID controller

PID (Proportional-Integral-Derivative) controller is the core of the feedback system. By comparing the current state $S(t)$ with the desired state, we get an error term $e(t)$. Using $e(t)$ as the input of PID controller, we can use the output $u(t)$ to drive the current state toward the desired state. The output expression is given by:

$$u(t) = K_p e(t) + K_i \int_0^t d\tau e(\tau) + K_d \frac{de(t)}{dt} \quad (9)$$

The Proportional term means that the larger the error is, the larger the output is to restore the state. The Integral term means that the longer the error persists, the larger the output is to further restore the state. The Derivative term means that the faster the error decreases, the smaller the output is to reduce oscillation. The three terms have individual coefficients (K_p, K_i, K_d). Their combinations vary widely across different situations.

For the PID of balancing pitch angle, the 3 coefficients need to be tuned along with the balanced pitch angle θ_0 . First, we found the approximate pitch angle. Based on this, we adjusted only θ_0 and K_p until the robots fell in both directions with a similar low probability, which means θ_0 is closed to the true balanced pitch angle. Then, we tuned K_d such that the robot wouldn't oscillate too much around the θ_0 . However, the robot may occasionally keep moving in one direction. We used large K_i to correct it. The 3 coefficients are very different. For example, $K_p = 48$, $K_i = 600$, and $K_d = 0.19$ with PID sample time = $2ms$. By tuning these coefficients and looking for a good balancing state, we can achieve better feedback performance.

3.4.3. Pulse width modulation

In order to control the robot smoothly and continuously, we need analog signals to do the task. However, the UNO has only digital output, which means the output voltage can be either HIGH level or LOW level without any values in between. PWM can solve this problem. Instead of encoding signals to voltage level, it produces a train of pulses as the output, and encodes signals to the width of each pulse. Meanwhile, our motor driver is able to receive the PWM signals.

However, the range of PWM values is limited. But this disadvantage can be ignored in our cases, since the resolution of PWM is much better than the resolution of geared motors.

3.5. Control Logic

The program for controlling runs in Arduino UNO. So far, we have realized these skills: connecting and initializing each component, reading angles and accelerations from IMU, using PWM to driver motors, balancing the robot by its pitch angle with PID, controlling the yaw angle (improving), changing the PID coefficients automatically for different situations (improving), controlling the linear motion (improving). The structure of control logic is based on 3.1 and Fig. 1, whose details are shown below:

3.5.1. Start

In this terminator, every component is being initialized. If anyone of them is not initialized successfully, the following controlling code would not run.

- `initImu()`: Sets the offsets of 3 axes for the gyroscope and the accelerometer, and waits until the IMU becomes stable.
- `initPID()`: Defines the balanced pitch and yaw, the PID coefficients for different situations, and the PID object for pitch and yaw.
- `initMotor()`: Uses `pinMode(pin, mode)` to initialize 2 EN pins and 4 IN pins.

3.5.2. Read States

In this process, we get the states of the robot from the calculations about quaternion. The logical code is shown in Code 1. For pitch and yaw angles, we calculate the error between the true angle and the balanced angle, unwrap the 2π period, and use the result as the input of feedback control. For accelerations, we compute the value in the world frame using the quaternion.

```
double unwrapAngle(float angle, double balanced) {
    double a = angle * 180 / M_PI - balanced;
    if (a < -180) { return a + 360; }
    else if (a > 180) { return a - 360; }
}

bool getRobotState() {
    imu.dmpGetQuaternion(&quaternion, imuFifoBuffer);
    imu.dmpGetYawPitchRoll(imuYpr, &quaternion, &gravity);
    pitchAngle = unwrapAngle(imuYpr[1], balancedPitch);
    yawAngle = unwrapAngle(imuYpr[0], balancedYaw);
    // ... doing similar jobs for acceleration ...
    imu.dmpGetLinearAccelInWorld(...);
}
```

Code 1. Example code of "Read States" process

3.5.3. Check States

The states gotten from the previous process is analyzed, and the next step is determined. For example (as shown in Code 2), the coefficients of PID controller will be changed to surviving mode if the pitch angle is too large, or the motors will be disabled if the robot is destined to fall.

```
if (abs(pitchAngle) > survivingPitch) {
    pitchPID.SetTunings(sKp, sKi, sKd);
} else { pitchPID.SetTunings(kp, ki, kd); }
// ...
if (abs(pitchAngle) > maxPitchAngle) { motorStop(); }
```

Code 2. Example code of "Check States" process

3.5.4. Feedback Control

Our feedback control is currently based on only PID controller. We use a library to manage the computing. In the code, the feedback

output can be gotten from `pid.Compute()`. The code is simple, while this process is essential to the robot. All the coefficients need plenty of time to tune manually.

3.5.5. Take Action

This process convert the digital signals from feedback control into a real action of the robots. For example, accelerating immediately, or rotating clockwise slowly. The action is also influenced by "Check States". As shown in Code 3, balancing the pitch is a priority over adjusting the yaw.

```
if (abs(motorValue) > maxMotorValue * 3) {
  motorMove(motorValue, ENA_PIN, IN1_PIN, IN2_PIN);
  motorMove(motorValue, ENB_PIN, IN3_PIN, IN4_PIN);
} else {
  motorMove(motorValue + yawCorrection, ENA_PIN, ...);
  motorMove(motorValue - yawCorrection, ENB_PIN, ...);
}
```

Code 3. Example code of "Take Action" process

REFERENCES

- [1] InvenSense, Inc., *Mpu-6000 and mpu-6050 product specification*, PS-MPU-6000A-00, version 3.4, Accessed: 2026-04-01, InvenSense (TDK Group), Aug. 2013. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>.
- [2] Arduino. "Arduino uno rev3." Accessed: 2026-04-01. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>.
- [3] Handson Technology, *L298n motor driver module datasheet*, Accessed: 2026-04-01, Handson Technology, n.d. [Online]. Available: <https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>.

4. PROJECT MANAGEMENT

4.1. Responsibilities

Initially, the project was divided into two parts: hardware and software. Lin was responsible for the hardware part, including electronic components and mechanical design. Yang was responsible for the software part, including signal processing and control logic. Our progress was unexpectedly fast. The first version of balancing robot was developed much earlier than planned.

After discussions, we both agreed that designing the hardware and software together is much more efficient than designing them individually and then combining them. By team working and peer learning, we quickly mastered both the hardware and software part. Therefore, we use a semi-parallel structure for our project. Both of us is responsible to design a robot with different skills individually while sharing the same mechanical and logical structure. There are also regular discussions about integrating these skills into one robot. Our current responsibilities about skill designing are listed below:

- Lin:
 - Locking skill: the robot can lock itself to a certain direction, even if it's rotated by us deliberately.
 - Adaption skill: the robot can judge the situation that it's facing or will face.
- Yang:
 - Motion skill: the robot can move forward and backward actively while keeping balanced.
 - Resurrection skill: the robot can restore itself from a heavy perturbation.

Combining these skills, we can achieve the advanced requirements mentioned in 2.2

4.2. Timeline

Timeline

4.3. Budget

Budget

5. CONCLUSION

Conclusion